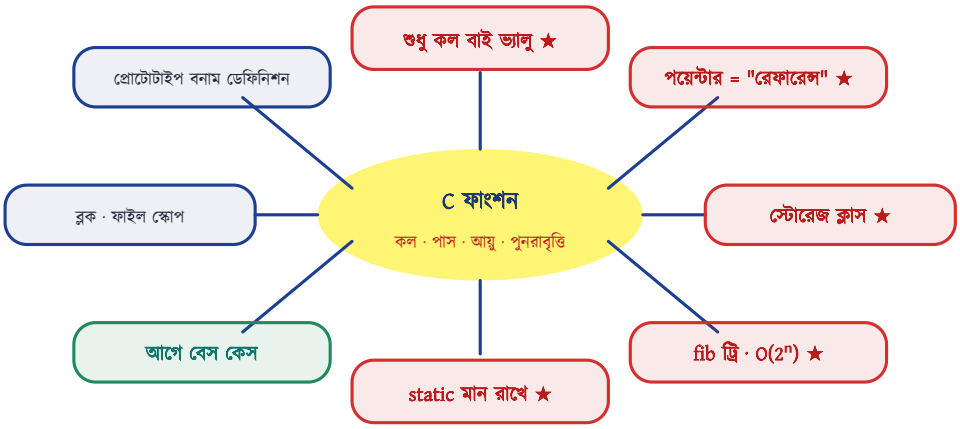
C-তে ফাংশন, স্কোপ ও রিকার্শন

ফাংশন · প্যারামিটার পাসিং · স্কোপ · স্টোরেজ ক্লাস · রিকার্শন: ১১ পাতায় পরীক্ষার পূর্ণ প্রস্তুতি



↑ পুরো লেসন এক পাতায়: লাল = বেশি আসে, সবুজ = যে নিয়মের ওপর প্রতিটি রিকার্শন দাঁড়িয়ে

সিলেবাসের যে অংশ

স্ট্রাকচার্ড প্রোগ্রামিং ল্যাঙ্গুয়েজ সিলেবাসে আছে: "**Functions and program structure: parameter passing conventions, scope rules and storage classes, recursion**"। ডেটা টাইপ, অপারেটর ও কন্ট্রোল স্ট্রাকচার আগের লেসনে; পয়েন্টার ও অ্যারে পরের লেসনে, স্ট্রাকচার ও ইউনিয়ন তার পরেরটিতে।

হাই-ইল্ড তালিকা

① প্রোটোটাইপ = ডিক্লারেশন; শেষে ;
② অ্যাকচুয়াল আর্গুমেন্ট → ফরমাল প্যারামিটারে কপি
③ C-তে **শুধু** কল বাই ভ্যালু
④ "রেফারেন্স" = পয়েন্টার পাঠানো
⑤ লোকাল ডিফল্টে `auto`: গারবেজ
⑥ `static`/গ্লোবাল ডিফল্টে ০
⑦ static লোকাল মান রাখে: 1 2 3
⑧ `register` ভেরিয়েবলে & নয়
⑨ `extern` ডিক্লেয়ার করে, ডিফাইন নয়
⑩ বেস কেস নেই → স্ট্যাক ওভারফ্লো

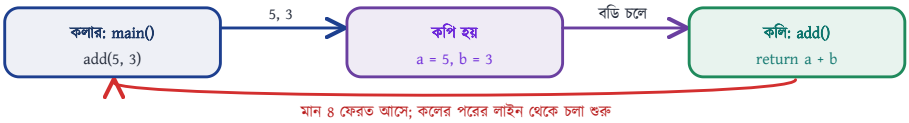
সংকেত: **হলুদ** = মূল শব্দ · **লাল বৃত্ত** = অবশ্যই মুখস্থ · **বেগুনি বক্স** = মনে রাখার কৌশল · **লাল বক্স** = পরীক্ষার ফাঁদ · **সবুজ** = সমাধানকৃত উদাহরণ

সূচিপত্র

✓ ফাংশন: প্রোটোটাইপ, ডেফিনিশন, কল ... পৃ. ২
✓ প্যারামিটার পাসিং: ভ্যালু বনাম অ্যাড্রেস ... পৃ. ৩
✓ স্কোপ, লিংকেজ, আয়ু ... পৃ. ৪
✓ স্টোরেজ ক্লাস: বড় টেবিল ... পৃ. ৫
✓ extern, static লিংকেজ, register ... পৃ. ৬
✓ রিকার্শন ও কল স্ট্যাক ... পৃ. ৭
✓ রিকার্শন ট্রি, জটিলতা, ইটারেশন ... পৃ. ৮
✓ আউটপুট ট্রেসিং অনুশীলন ... পৃ. ৯
✓ রাতের পরীক্ষার পাতা ... পৃ. ১০
✓ অনুশীলনী MCQ + লিখিত ... পৃ. ১১

১ ফাংশন: প্রোটোটাইপ, ডেফিনিশন, কল

সংজ্ঞা: **ফাংশন (Function)** হলো নামযুক্ত, স্বয়ংসম্পূর্ণ কোড ব্লক, যা একটি কাজ করে এবং বহুবার কল করা যায়। একটি C প্রোগ্রাম কতগুলো ফাংশনের সমষ্টি; এক্সিকিউশন শুরু হয় `main` থেকে। সাধারণ রূপ: `return_type name(type p1, type p2) { body; return value; }`। সুবিধা: **পুনঃব্যবহার** ও **মডুলারিটি**। দুই প্রকার: **লাইব্রেরি** ফাংশন (`printf`, `sqrt`) ও **ইউজার-ডিফাইনড** ফাংশন।



একটি কল: আর্গুমেন্ট ঢোকে, একটি মান ফেরত আসে

```c
#include <stdio.h>
int add(int a, int b);   /* prototype */

int main(void) {
    int s = add(5, 3);   /* call */
    printf("%d\n", s);   /* 8 */
    return 0;            /* success */
}

int add(int a, int b) {  /* definition */
    return a + b;
}
```

পরিভাষা	অর্থ
প্রোটোটাইপ	ডিক্লারেশন: নাম, রিটার্ন টাইপ, প্যারামিটার টাইপ; শেষে `;`, বডি নেই
ডেফিনিশন	হেডার + বডি; প্রতি ফাংশনে ঠিক **একটি**
কল	`add(5, 3)`: ভেতরে যায়, একটি মান ফেরত পায়
অ্যাকচুয়াল প্যারামিটার	কলে দেওয়া আর্গুমেন্ট: 5, 3
ফরমাল প্যারামিটার	হেডারের ভেরিয়েবল: *a*, *b*

নিয়ম	যা মনে রাখতে হবে
রিটার্ন মান	প্রতি কলে সর্বোচ্চ **একটি** মান; বেশি লাগলে পয়েন্টার প্যারামিটার বা `struct` ফেরত
`void` ফাংশন	কিছু ফেরত দেয় না; আগেভাগে বেরোতে শুধু `return;` লেখা যায়
`main`	`int` ফেরত দেয়: `0` = সফল; `void main()` মানসম্মত নয়
প্রোটোটাইপ	নাম ঐচ্ছিক (`int add(int, int);`); একেবারে না থাকলে: C89 `int` ধরত, C99+ নিষেধ

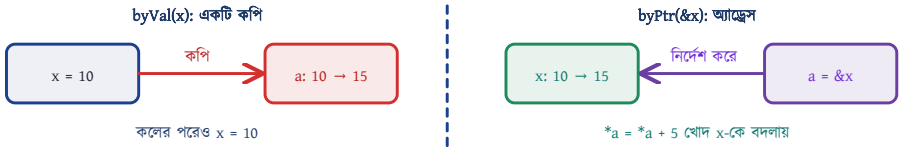
মনে রাখার কৌশল: "**প্রোটোটাইপ = ভিজিটিং কার্ড; ডেফিনিশন = মানুষটি**।" কার্ড অনেক বিলানো যায়, মানুষ কিন্তু একজনই।

পরীক্ষার ফাঁদ: আর্গুমেন্ট মূল্যায়নের ক্রম **অনির্দিষ্ট**: `f(i++, i++)` আনডিফাইনড বিহেভিয়ার। স্ট্যান্ডার্ড C-তে এক ফাংশনের ভেতরে আরেকটি **ডিফাইন** করা যায় না।

সমাধানকৃত উদাহরণ ✍️ আউটপুট: `int sq(int x){return x*x;}` → `printf("%d", sq(3) + sq(4));` ছাপে **25**। অ্যাকচুয়াল 3, 4; ফরমাল *x*।

২ প্যারামিটার পাসিং: ভ্যালু ও অ্যাড্রেস দিয়ে

সংজ্ঞা: **কল বাই ভ্যালু** (Call by Value)-তে কলি ফাংশন প্রতিটি আর্গুমেন্টের একটি **কপি** পায় তার ফরমাল প্যারামিটারে, তাই ভেতরের পরিবর্তন কলারের কাছে পৌঁছায় না। C **সব আর্গুমেন্ট ভ্যালুতে** পাঠায়। কলারের ভেরিয়েবল বদলাতে হলে তার **অ্যাড্রেস** পাঠাতে হয়: ফরমাল প্যারামিটার হয় পয়েন্টার, আর ফাংশন `*p` দিয়ে লেখে। একে সাধারণত **কল বাই রেফারেন্স** (Call by Reference) বলা হয়, কিন্তু C-তে এটি **অনুকরণ** মাত্র — অ্যাড্রেসটিই কপি হয়। আসল রেফারেন্স (`int &a`) শুধু C++-এ।



বাঁয়ে: ফাংশন কাজ করে ফটোকপিতে · ডানে: তার হাতে x-এর চাবি

```
void byVal(int a)  { a = a + 5; }
void byPtr(int *a) { *a = *a + 5; }

int x = 10;
byVal(x);    /* x is still 10 */
byPtr(&x);   /* x becomes 15  */
```

```
void swap(int *p, int *q) {
    int t = *p;  *p = *q;  *q = t;
}
int a = 10, b = 20;
swap(&a, &b);  /* a = 20, b = 10 */
```

বিষয়	কল বাই ভ্যালু	অ্যাড্রেস (পয়েন্টার) দিয়ে	C++ রেফারেন্স
কী পাঠানো হয়	মানের কপি	**অ্যাড্রেসের** কপি	ভেরিয়েবলের অ্যালিয়াস
ফরমাল / কল	`int a` / `f(x)`	`int *a` / `f(&x)`	`int &a` / `f(x)`
কলারের ভেরিয়েবল বদলায়?	না	হ্যাঁ, `*a` দিয়ে	হ্যাঁ
C-তে আছে?	**একমাত্র** পদ্ধতি	হ্যাঁ (পয়েন্টারের মান)	না — শুধু C++

অ্যারে পয়েন্টারে রূপ নেয় (decay): `void f(int a[])` ≡ `void f(int *a)`: শুধু ০ নম্বর উপাদানের অ্যাড্রেস যায়, অ্যারে **কপি হয় না**, ভেতরের পরিবর্তন কলার দেখে। *f*-এর ভেতরে `sizeof(a)` = **পয়েন্টারের** আকার, তাই দৈর্ঘ্যও পাঠাও। বিস্তারিত: পরের লেসন।

স্ট্রাকচার কপি হয়: `struct` আর্গুমেন্ট পুরোটা কপি হয় — বড় হলে ধীর, তাই `const struct T *p` পাঠাও; `const` প্রতিশ্রুতি দেয় যে ডেটা বদলাবে না। স্ট্রাকচার বিস্তারিত: লেসন ৪।

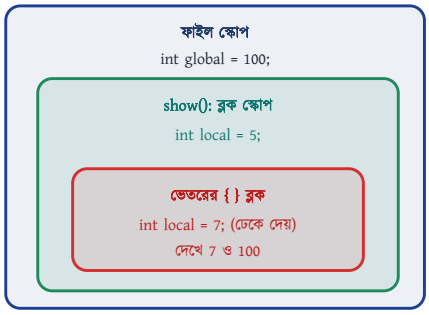
মনে রাখার কৌশল: "**ভ্যালু = ফটোকপি; অ্যাড্রেস = বাড়ির চাবি।**" ফটোকপিতে দাগ দিলে আসলটা নিরাপদ; চাবি পেলে আসল বাড়িই এলোমেলো।

প্রচলিত ভুল ধারণা: "C-তে কল বাই রেফারেন্স আছে" — না: পয়েন্টারও কপি হয়। কলির ভেতরে `p = &y;` লিখলে কলারের পয়েন্টার **সরে না**।

সমাধানকৃত উদাহরণ ✍️ ভ্যালুতে swap: `void swap(int a,int b) {int t=a;a=b;b=t;}, x=10,y=20; swap(x,y); printf("%d %d",x,y);` → **10 20**।

৩ স্কোপের নিয়ম, লিংকেজ ও আয়ু

সংজ্ঞা: কোনো আইডেন্টিফায়ারের **স্কোপ (Scope)** হলো প্রোগ্রাম টেক্সটের সেই অংশ, যেখানে তার নাম ব্যবহার করা যায়। **লোকাল** (ব্লকের ভেতরে, বা প্যারামিটার) ভেরিয়েবলের **ব্লক স্কোপ**; **গ্লোবাল** (সব ফাংশনের বাইরে) ভেরিয়েবলের **ফাইল স্কোপ**, ফাইলের শেষ পর্যন্ত। C স্ট্যান্ডার্ডে **চার ধরনের স্কোপ**: ব্লক, ফাইল, ফাংশন ও ফাংশন প্রোটোটাইপ।



ভেতরের বাক্স বাইরে দেখে, বাইরেরটা ভেতরে নয়

```c
int global = 100;          /* file scope */
void show(void) {
    int local = 5;         /* block scope */
    {
        int local = 7;     /* shadows it  */
        printf("%d ", local);       /* 7 */
    }
    printf("%d %d", local, global);
}                          /* prints 7 5 100 */
/* in main(): local is unknown → error */
```

স্কোপ (C স্ট্যান্ডার্ড)	কার জন্য	কোথা থেকে ... কোথা পর্যন্ত দৃশ্যমান
ব্লক	লোকাল, ফাংশন প্যারামিটার	ডিক্লারেশনের স্থান → সমাপ্তি }
ফাইল	গ্লোবাল, ফাংশনের নাম	ডিক্লারেশনের স্থান → সোর্স ফাইলের শেষ
ফাংশন	**শুধু লেবেল** (goto-র জন্য)	পুরো ফাংশন, লেবেলের আগেও
ফাংশন প্রোটোটাইপ	প্রোটোটাইপের প্যারামিটার নাম	সেই প্রোটোটাইপের শেষ পর্যন্ত

পরিভাষা	যে প্রশ্নের উত্তর দেয়	মান
স্কোপ	নাম কোথায় ব্যবহার করা যায়? (কম্পাইল টাইম)	ব্লক, ফাইল, ফাংশন, প্রোটোটাইপ
আয়ু (Lifetime)	স্টোরেজ কতক্ষণ থাকে? (রান টাইম)	অটোমেটিক (ব্লক), স্ট্যাটিক (পুরো প্রোগ্রাম), অ্যালোকেটেড (`malloc`...`free`)
লিংকেজ (Linkage)	অন্য ফাইলে একই নাম কি একই অবজেক্ট?	**এক্সটার্নাল**, **ইন্টারনাল**, নেই

মনে রাখার কৌশল: "**স্কোপ = যে ঘরে নাম ধরে ডাকা যায়; আয়ু = মানুষটি কতদিন বাঁচে।**" static লোকাল সারা প্রোগ্রাম বাঁচে, কিন্তু সাড়া দেয় শুধু নিজের ঘরে।

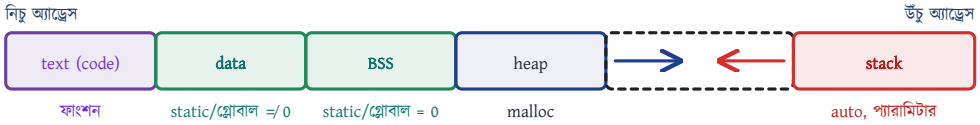
পরীক্ষার ফাঁদ: ঢাকা পড়া (shadowed) গ্লোবালকে সেই ব্লকে নাম দিয়ে ধরা যায় না: C-তে `::x` নেই (শুধু C++)। ডিক্লারেশনের লাইনের আগে ব্যবহার চলে না।

সমাধানকৃত উদাহরণ ✍️ আউটপুট: `int x=1; void f(){int x=2; {x=3;} printf("%d",x);}` ছাপে 3: ভেতরের ব্লক f-এর x-এ লেখে; গ্লোবাল থাকে 1।

8 স্টোরেজ ক্লাস: বড় টেবিল

সংজ্ঞা: **স্টোরেজ ক্লাস (Storage Class)** স্পেসিফায়ার কম্পাইলারকে ভেরিয়েবলের **সংরক্ষণের স্থান**, **ডিফল্ট প্রাথমিক মান**, **স্কোপ**, **আয়ু** ও **লিংকেজ** জানায়। C-তে চারটি প্রচলিত: `auto`, `register`, `static`, `extern` (C11-এ `_Thread_local`; `typedef` শুধু ব্যাকরণগতভাবে)। কিওয়ার্ড ছাড়া লোকাল ভেরিয়েবল **auto**; কিওয়ার্ড ছাড়া গ্লোবালের লিংকেজ এক্সটার্নাল।

ক্লাস	কোথায় থাকে	ডিফল্ট	স্কোপ	আয়ু	লিংকেজ
auto	স্ট্যাক	গারবেজ	ব্লক	ব্লক	নেই
register	CPU রেজিস্টার (পেলে)	গারবেজ	ব্লক	ব্লক	নেই
static লোকাল	data / BSS	**0**	ব্লক	পুরো প্রোগ্রাম	নেই
static গ্লোবাল	data / BSS	0	ফাইল	পুরো প্রোগ্রাম	ইন্টারনাল
extern	যেখানে ডিফাইন করা	0 (ডেফিনিশন থেকে)	ফাইল (বা ব্লক)	পুরো প্রোগ্রাম	এক্সটার্নাল
সাধারণ গ্লোবাল	data / BSS	0	ফাইল	পুরো প্রোগ্রাম	এক্সটার্নাল



সাধারণ মেমোরি বিন্যাস: কোন স্টোরেজ ক্লাস কোথায় থাকে

```
void counter(void) {
    static int count = 0;  /* once */
    count++;
    printf("%d ", count);
}
/* main: counter(); counter(); counter();
   prints: 1 2 3                          */
/* without static:  1 1 1                 */
```

static লোকাল: তিনটি তথ্য: ① এটি তৈরি ও **একবারই ইনিশিয়ালাইজ** হয়, `main` শুরুর আগে — প্রতি কলে `static int count = 0;` লাইন আবার চলে **না**। ② এর ইনিশিয়ালাইজার **ধ্রুবক রাশি** হতে হবে: `static int n = f();` C-তে কম্পাইল এরর। ③ কলের মাঝে মান ধরে রাখে, তবু **স্কোপ ব্লকেই**: `main` `count`-কে নাম ধরে পায় না। ফাংশন এর অ্যাড্রেস নিরাপদে ফেরত দিতে পারে — এটি কখনো মরে না।

মনে রাখার কৌশল: "**A-R অল্পায়ু, S-E দীর্ঘায়ু**।" Auto ও Register ব্লক শেষে মরে, শুরু গারবেজে; Static ও Extern পুরো রান বাঁচে, শুরু শূন্যে।

পরীক্ষার ফাঁদ: শূন্য ডিফল্ট = শুধু **স্ট্যাটিক স্টোরেজ** (static, গ্লোবাল)। ইনিশিয়ালাইজ না করা `auto` ও `malloc` মেমোরিতে গারবেজ (`calloc` দেয় 0)।

সমাধানকৃত উদাহরণ ✍️ আউটপুট: `int f(){static int s=5; s+=2; return s;}` → `f(); f(); printf("%d", f());` ছাপে **11** (7, 9, 11)।

৫ extern, static লিংকেজ ও register

সংজ্ঞা: `extern int g;` একটি **ডিক্লারেশন (Declaration)**: এটি বলে "g আছে, অন্য কোথাও ডিফাইন করা" এবং **কোনো স্টোরেজ বরাদ্দ করে না**। সব ফাংশনের বাইরে একমাত্র **ডেফিনিশন (Definition)** `int g = 5;` মেমোরি বরাদ্দ করে ও শূন্য ডিফল্ট বহন করে। প্রোগ্রাম g-কে বহুবার ডিক্লেয়ার করতে পারে, কিন্তু ডিফাইন **ঠিক একবার**; **লিংকার** প্রতিটি `extern`-কে সেই এক ডেফিনিশনের সঙ্গে জোড়ে।



extern ফাইল পেরিয়ে পৌঁছায়; ফাংশনের বাইরে static দরজা বন্ধ করে

কিওয়ার্ড ও স্থান	প্রভাব
লোকাল ভেরিয়েবলে `static`	আয়ু হয় পুরো প্রোগ্রাম; স্কোপ অপরিবর্তিত (ব্লক); লিংকেজ নেই
গ্লোবাল ভেরিয়েবলে `static`	**ইন্টারনাল লিংকেজ:** শুধু নিজের সোর্স ফাইলে দৃশ্যমান
ফাংশনে `static`	ফাংশনটি নিজের ফাইলে সীমাবদ্ধ (হেল্পার ফাংশন, নামের সংঘাত এড়ায়)
ভেরিয়েবলে `extern`	শুধু ডিক্লারেশন; বাইরের ডেফিনিশনকে নির্দেশ করে
ফাইল স্কোপে `extern int g = 5;`	ইনিশিয়ালাইজার থাকায় এটি **ডেফিনিশন** (বৈধ; কম্পাইলার সতর্ক করে)
কিওয়ার্ডহীন ফাংশন	ডিফল্টে `extern`: প্রোটোটাইপ দিয়ে অন্য ফাইল থেকে কল করা যায়

register: গতির জন্য ভেরিয়েবলকে CPU রেজিস্টারে রাখার **অনুরোধ** (লুপ কাউন্টার)। কম্পাইলার **উপেক্ষা করতে পারে** (আধুনিক অপটিমাইজার নিজেই রেজিস্টার বাছে)। শুধু লোকাল ও প্যারামিটারে; এর **& নেওয়া যায় না** — মেমোরিতে থাকলেও কম্পাইল এরর; ডিফল্ট মান গারবেজ।

```
void loop(void) {
    register int i;
    for (i = 0; i < 1000; i++)
        ;            /* fast counter  */
    int *p = &i;     /* ✗ error: address
                        of register   */
}
register int r;  /* ✗ not at file scope */
```

প্রচলিত ভুল ধারণা — সংশোধিত: ① "`extern` ভেরিয়েবল তৈরি করে, মান 0" — না: স্টোরেজ ও শূন্য ডিফল্ট অন্যত্র থাকা ডেফিনিশনের। ② "`static` গ্লোবাল যেকোনো জায়গায় ব্যবহারযোগ্য" — না: ফাংশনের বাইরে এর মানে **ফাইলে সীমাবদ্ধ**। ③ "`register` রেজিস্টারের নিশ্চয়তা দেয়" — না: এটি শুধু ইঙ্গিত। ④ "register ভেরিয়েবলে `&r` চলে" — কম্পাইল এরর। ⑤ দুই ফাইলেই `int g = 1;` → **মাল্টিপল ডেফিনিশন** লিংক এরর।

মনে রাখার কৌশল: "**extern = 'অন্য কোথাও থাকে'; বাইরের static = 'প্রবেশ নিষেধ' সাইনবোর্ড।**" ডিক্লারেশন = গুজব; ডেফিনিশন = জন্ম।

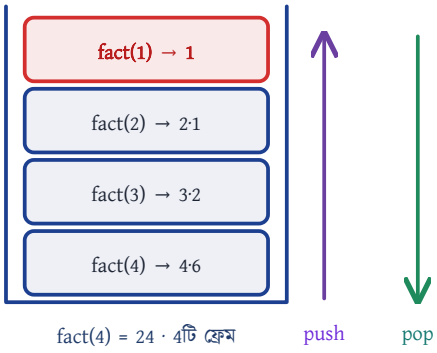
সমাধানকৃত উদাহরণ ✍️ বৈধ?: একটি ফাইলে: `extern int n; extern int n; int n = 3;` → **বৈধ:** দুটি ডিক্লারেশন, একটি ডেফিনিশন।

৬ রিকার্শন ও কল স্ট্যাক

সংজ্ঞা: **রিকার্শন (Recursion)**: ফাংশন নিজেকে কল করে, **প্রত্যক্ষভাবে** (f → f) বা **পরোক্ষভাবে** (f → g → f)। প্রতিটি রিকার্সিভ ফাংশনে লাগে একটি **বেস কেস (Base Case)**, যা আর রিকার্স না করে ফেরত দেয়, এবং একটি **রিকার্সিভ কেস**, যা বেস কেসের দিকে এগোয়। প্রতিটি কল পায় নিজস্ব **স্ট্যাক ফ্রেম (Stack Frame)** (অ্যাক্টিভেশন রেকর্ড: প্যারামিটার, লোকাল, রিটার্ন অ্যাড্রেস)। বেস কেসে পৌঁছানো না গেলে ফ্রেম জমতে জমতে **স্ট্যাক ওভারফ্লো** (রান-টাইম ক্র্যাশ)।

```
long fact(int n) {
    if (n == 0 || n == 1)      /* base case */
        return 1;
    return n * fact(n - 1);    /* recursive */
}
/* fact(5) = 5*4*3*2*1 = 120 */

int fib(int n) {               /* 0 1 1 2 3 5 8 */
    if (n <= 1) return n;      /* fib(0)=0, fib(1)=1 */
    return fib(n - 1) + fib(n - 2);
}
```



নামার সময় জমে, ওঠার সময় খোলে

প্রকার	অর্থ	উদাহরণ
প্রত্যক্ষ (Direct)	ফাংশন নিজেকে কল করে	`fact`, `fib`
পরোক্ষ (Indirect)	A কল করে B-কে, B কল করে A-কে	`isEven` ↔ `isOdd`
টেইল (Tail)	রিকার্সিভ কলটিই **শেষ** কাজ; পরে আর কিছু বাকি নেই	`return gcd(b, a % b);`
নন-টেইল (Head)	কল ফেরার পরও কাজ বাকি থাকে	`return n * fact(n - 1);`
ট্রি (Multiple)	প্রতি ফ্রেমে একাধিক রিকার্সিভ কল	`fib`, টাওয়ার অব হ্যানয়

```
int isOdd(int n);          /* prototype */
int isEven(int n) {
    return n == 0 ? 1 : isOdd(n - 1);
}
int isOdd(int n) {
    return n == 0 ? 0 : isEven(n - 1);
}          /* isEven(4) → 1  (indirect) */
```

```
long factT(int n, long acc) {
    if (n <= 1) return acc;
    return factT(n - 1, n * acc); /* tail */
}
/* factT(5, 1) = 120: the frame has no
   pending work, so an optimiser can
   reuse it (a loop in disguise)      */
```

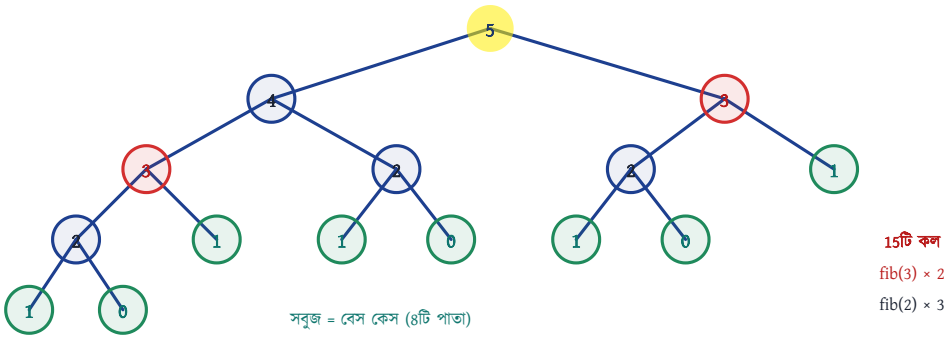
মনে রাখার কৌশল: "**রাশিয়ান পুতুল**": নামার পথে পুতুল খোলো (কল), সবচেয়ে ছোটটি নিরেট (বেস কেস), ওঠার পথে বন্ধ করো (রিটার্ন)।

পরীক্ষার ফাঁদ: টেইল-কল অপটিমাইজেশন কম্পাইলার **করতে পারে** (যেমন gcc -O2); C স্ট্যান্ডার্ড বাধ্য করে না। C-তে `main` নিজেকে কল করতে পারে (C++-এ নয়)।

সমাধানকৃত উদাহরণ ✍ গভীরতা: `fact(5)` → n = 5, 4, 3, 2, 1-এর ফ্রেম: সর্বোচ্চ গভীরতা **5**; ফল 120। `fact(-1)` বেস কেসে পৌঁছায় না → ওভারফ্লো।

৭ রিকার্শন ট্রি, জটিলতা, রিকার্শন বনাম ইটারেশন

সংজ্ঞা: **রিকার্শন ট্রি (Recursion Tree)** প্রতিটি কলের জন্য একটি নোড আঁকে। সাধারণ `fib(n)` প্রতি ফ্রেমে দুটি কল করে এবং একই মান বারবার হিসাব করে: কলসংখ্যা $C(n) = C(n-1) + C(n-2) + 1$, $C(0) = C(1) = 1$, যার সমাধান $C(n) = 2 \cdot F(n+1) - 1$। `fib(5)`-এর জন্য: $2 \cdot 8 - 1 = 15$টি কল। সময় $O(2^n)$ (সূক্ষ্ম: $\Theta(\varphi^n)$, $\varphi \approx 1.618$); স্ট্যাকের গভীরতা মাত্র n।



নোড n = একটি কল fib(n); লাল fib(3) দুবার হিসাব হয় — অপচয়

n	0	1	2	3	4	5	6
fib(n)	0	1	1	2	3	5	8
সাধারণ fib(n)-এর কলসংখ্যা	1	1	3	5	9	**15**	25

অ্যালগরিদম	সময়	স্ট্যাক স্পেস
ফ্যাক্টরিয়াল, রিকার্সিভ	$O(n)$	$O(n)$
ফ্যাক্টরিয়াল, লুপ	$O(n)$	$O(1)$
fib, সাধারণ রিকার্সিভ	**$O(2^n)$**	$O(n)$
fib, মেমোয়াইজড	$O(n)$	$O(n)$
GCD (ইউক্লিড)	$O(\log \min(a,b))$	$O(\log)$

```
long memo[50];   /* static: all 0 */
long fibm(int n) {
    if (n <= 1) return n;
    if (memo[n]) return memo[n];
    return memo[n] =
        fibm(n - 1) + fibm(n - 2);
}         /* each n solved once */
```

বিষয়	রিকার্শন	ইটারেশন (লুপ)
থামে	বেস কেসে	লুপ শর্ত মিথ্যা হলে
মেমোরি	প্রতি সক্রিয় কলে একটি স্ট্যাক ফ্রেম	ধ্রুবক
গতি · ব্যর্থতা	ধীর (কলের ওভারহেড) · স্ট্যাক ওভারফ্লো	দ্রুত · অসীম লুপ
রূপান্তরযোগ্য?	হ্যাঁ: যেকোনো রিকার্শন → লুপ (+ স্পষ্ট স্ট্যাক); ট্রি, হ্যানয়ে রিকার্শন বেশি পরিষ্কার	

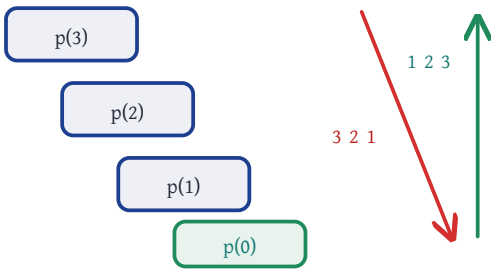
মনে রাখার কৌশল: **"গভীরতা = স্ট্যাক, নোড = সময়।"** সাধারণ fib: n গভীর, প্রায় 2^n চওড়া।

সমাধানকৃত উদাহরণ ✍️ হ্যানয়: ৩টি চাকতিতে $2^3 - 1 = 7$টি চাল; ৪টিতে 15; প্রতিটি নতুন চাকতি দ্বিগুণ করে এক যোগ।

৮ আউটপুট ট্রেসিং অনুশীলন

পদ্ধতি: যেকোনো রিকার্সিভ আউটপুট প্রশ্নে: ① প্রতিটি আর্গুমেন্টসহ নিচের দিকে **কল চেইন** লেখো; ② কোথায় **বেস কেস** কাজ করে চিহ্নিত করো; ③ এক ফ্রেম করে **রিটার্ন মান ওপরে** নিয়ে যাও; ④ রিকার্সিভ কলের **আগের** `printf` নামার পথে ছাপে, **পরের**টি ওঠার পথে (উল্টো ক্রমে)।

```
void p(int n) {
    if (n == 0) return;
    printf("%d", n);   /* before: down */
    p(n - 1);
    printf("%d", n);   /* after: up    */
}
p(3);              /* prints 321123 */
```



নামতে ছাপে 3 2 1, উঠতে 1 2 3

① অঙ্কের যোগফল:

```
int sd(int n) {
  return n == 0 ? 0 : n % 10 + sd(n / 10);
}
```

`sd(1234)` = 4 + 3 + 2 + 1 + 0 = **10**; ৫টি কল (1234, 123, 12, 1, 0)।

② ঘাত (Power):

```
int pw(int b, int e) {
  return e == 0 ? 1 : b * pw(b, e - 1);
}
```

`pw(2, 10)` = 1024, ১১টি কলে। e অর্ধেক করলে (`pw(b*b, e/2)`) হয় $O(\log e)$।

③ GCD (ইউক্লিড):

```
int gcd(int a, int b) {
  return b == 0 ? a : gcd(b, a % b);
}
```

`gcd(48, 18)`: (48,18) → (18,12) → (12,6) → (6,0) → **6**। LCM = 48·18 / 6 = 144।

④ দুই ধাপে নামা:

```
int f(int n) {
  return n <= 1 ? 1 : n * f(n - 2);
}
```

`f(7)` = 7·5·3·1 = **105**, 7! = 5040 নয়: আর্গুমেন্ট ২ করে কমে।

⑤ রিকার্শনে বাইনারি:

```
void bin(int n) {
  if (n > 1) bin(n / 2);
  printf("%d", n % 2);
}
```

`bin(10)` ছাপে **1010**: অঙ্কগুলো বেরোয় ওঠার পথে।

⑥ রিকার্শনের ভেতরে static:

```
int g(int n) {
  static int s = 0;
  s += n;  if (n > 0) g(n - 1);
  return s;
}
```

`g(3)` → একটিই শেয়ার্ড s: 3+2+1+0 = **6**; দ্বিতীয়বার `g(3)` দেয় 12।

মনে রাখার কৌশল: "**আগে প্রিন্ট = সিঁড়ি দিয়ে নামা; পরে প্রিন্ট = আবার ওঠা।**" নামা সোজা পড়ে, ওঠা উল্টো।

পরীক্ষার ফাঁদ: `static` লোকাল **সব ফ্রেমে শেয়ার্ড** — একটিই কপি, প্রতি কলে আলাদা নয়। `auto` লোকাল প্রতিটি ফ্রেমে নতুন কপি পায়।

সমাধানকৃত উদাহরণ ✍️ যোগফল: `int s(int n){return n==0?0:n+s(n-1);}` → `s(100)` = **5050**, ১০১টি কল গভীর।

পরীক্ষার আগের রাতের পাতা

তথ্য-পাতা:

প্রোটোটাইপ: `int add(int, int);`

ডেফিনিশন: ঠিক একটি, বডিসহ

অ্যাকচুয়াল = কলে; ফরমাল = হেডারে

`main` ফেরত দেয় `int`; 0 = সফল

C: শুধু কল বাই ভ্যালু

"রেফারেন্স" = `f(&x)` + `int *p`

অ্যারে প্যারামিটার = পয়েন্টার; দৈর্ঘ্য পাঠাও

৪ স্কোপ: ব্লক, ফাইল, ফাংশন, প্রোটোটাইপ

auto/register: গারবেজ, ব্লক আয়ু

static/গ্লোবাল: 0, পুরো-প্রোগ্রাম আয়ু

static গ্লোবাল/ফাংশন → ইন্টারনাল লিংকেজ

`extern`: ডিক্লারেশন, স্টোরেজ নেই

register-এ & নয়; শুধু ইঙ্গিত

fib(5) = 5, ১৫টি কলে; $2 \cdot F(n+1) - 1$

MCQ-র জন্য সেরা ১২টি এক-লাইনার

১. লোকালের ডিফল্ট স্টোরেজ ক্লাস: `auto`
২. static লোকাল: একবার ইনিশিয়ালাইজ, মান রাখে
৩. static-সহ counter() ×৩ → 1 2 3
৪. ভ্যালুতে swap → পরিবর্তন নেই (10 20)
৫. লেবেলের স্কোপ ফাংশন স্কোপ
৬. শ্যাডোয়িং: ভেতরের নাম বাইরেরটাকে ঢাকে
৭. রিকার্শনে বেস কেস লাগে
৮. প্রতিটি কল = একটি স্ট্যাক ফ্রেম
৯. বেস কেস নেই → স্ট্যাক ওভারফ্লো
১০. fact O(n); সাধারণ fib $O(2^n)$; মেমো O(n)
১১. টেইল কল = রিকার্সিভ কলই শেষ কাজ
১২. হ্যানয়: $2^n - 1$ চাল

প্রধান ফাঁদ: C-তে আসল কল বাই রেফারেন্স নেই · `extern` কিছু বরাদ্দ করে না · static গ্লোবাল ফাইলে সীমাবদ্ধ · `register` উপেক্ষিত হতে পারে · `f(i++, i++)` আনডিফাইনড · static লোকাল সব ফ্রেমে শেয়ার্ড · অ্যারে প্যারামিটারের `sizeof` = পয়েন্টারের আকার।

সব কৌশল একসঙ্গে: ভিজিটিং কার্ড বনাম মানুষ · ফটোকপি বনাম বাড়ির চাবি · ঘর বনাম আয়ু · A-R অল্পায়ু, S-E দীর্ঘায়ু · extern = অন্যত্র থাকে, static = প্রবেশ নিষেধ · রাশিয়ান পুতুল · গভীরতা = স্ট্যাক, নোড = সময় · সিঁড়ি দিয়ে নামা, আবার ওঠা।



তিন ধারণা, তিন লাইন

সমাধানকৃত উদাহরণ ✍️ একটি কিওয়ার্ডে আউটপুট বদলায়: `void c(){ int n = 0; n++; printf("%d ", n); }` তিনবার কল করলে ছাপে "1 1 1"। এর বদলে `static int n = 0;` লিখলে একই তিন কলে ছাপে "1 2 3" — ইনিশিয়ালাইজার একবারই চলে, কলের মাঝে মান টিকে থাকে।

✓ অনুশীলনী MCQ (পরীক্ষার ধাঁচে, ১০ নম্বর)

১. C-তে লোকাল ভেরিয়েবলের ডিফল্ট স্টোরেজ ক্লাস:
ক) static খ) extern
গ) auto ঘ) register

২. `counter()`-এ আছে `static int c = 0; c++;` `printf("%d ", c);`। তিনটি কল ছাপে:
ক) 1 2 3 খ) 1 1 1
গ) 0 1 2 ঘ) 3 3 3

৩. C-তে ফাংশনের আর্গুমেন্ট পাঠানো হয়:
ক) শুধু রেফারেন্সে খ) নাম দিয়ে
গ) ভ্যালু ও রেফারেন্স দুভাবেই ঘ) শুধু ভ্যালুতে (পয়েন্টার রেফারেন্সের অনুকরণ করে)

৪. `static int`-এর ডিফল্ট প্রাথমিক মান:
ক) গারবেজ খ) 0
গ) 1 ঘ) -1

৫. অ্যাড্রেস অপারেটর `&` প্রয়োগ করা যায় না:
ক) register ভেরিয়েবলে খ) static ভেরিয়েবলে
গ) extern ভেরিয়েবলে ঘ) গ্লোবাল ভেরিয়েবলে

৬. গ্লোবাল ভেরিয়েবলে `static` দিলে এটি পায়:
ক) এক্সটার্নাল লিংকেজ খ) ব্লক স্কোপ
গ) ইন্টারনাল লিংকেজ (ফাইলে সীমাবদ্ধ) ঘ) অটোমেটিক আয়ু

৭. প্রথমটি ধরে সাধারণ রিকার্সিভ `fib(5)` কয়টি কল করে?
ক) 5 খ) 8
গ) 9 ঘ) 15

৮. `p(n)` n ছাপে, `p(n-1)` কল করে, আবার n ছাপে; `p(0)` ফেরত যায়। `p(3)` ছাপে:
ক) 123321 খ) 321123
গ) 321 ঘ) 321321

৯. ফাইল স্কোপে লেখা `extern int x;`:
ক) স্টোরেজ ছাড়া x ডিক্লেয়ার খ) x ডিফাইন করে 0 বসায় করে
গ) x-কে ফাইলে সীমাবদ্ধ করে ঘ) স্ট্যাকে x বরাদ্দ করে

১০. সাধারণ রিকার্সিভ ফিবোনাচির টাইম কমপ্লেক্সিটি:
ক) $O(n)$ খ) $O(\log n)$
গ) $O(n^2)$ ঘ) $O(2^n)$

উত্তরমালা → ১-গ · ২-ক · ৩-ঘ · ৪-খ · ৫-ক · ৬-গ · ৭-ঘ · ৮-খ · ৯-ক · ১০-ঘ

লিখিত প্রশ্ন: একটি টেবিলসহ (স্টোরেজ, ডিফল্ট মান, স্কোপ, আয়ু) C-এর চারটি স্টোরেজ ক্লাস ব্যাখ্যা করো। `static` লোকাল যে মান ধরে রাখে তা দেখাতে একটি প্রোগ্রাম এবং একটি রিকার্সিভ ফ্যাক্টরিয়াল ফাংশন লেখো; `fact(4)`-এর স্ট্যাক ট্রেস করো। ১০ নম্বর

১ **টেবিল (৪)** — auto: স্ট্যাক, গারবেজ, ব্লক, ব্লক · register: পেলে CPU রেজিস্টার, গারবেজ, ব্লক, ব্লক, `&` নয় · static: data/BSS, 0, ব্লক বা ফাইল, পুরো প্রোগ্রাম · extern: অন্যত্র থাকা ডেফিনিশনের ডিক্লারেশন, 0, ফাইল, পুরো প্রোগ্রাম। ৪

২ **static প্রোগ্রাম (২)** — `static int count = 0; count++;`-সহ `counter()` তিনবার কল করলে ছাপে 1 2 3; `static` ছাড়া 1 1 1। ২

৩ **ফ্যাক্টরিয়াল (২)** — বেস কেস `n == 0 || n == 1` হলে 1; নইলে `n * fact(n - 1)`। ২

৪ **ট্রেস (২)** — push fact(4), fact(3), fact(2), fact(1); fact(1) = 1; pop: 2·1 = 2, 3·2 = 6, 4·6 = 24। ২

পরীক্ষার আগে ✓

- ✓ প্রোটোটাইপ, ডেফিনিশন ও কল আলাদা করতে পারি
- ✓ পয়েন্টার দিয়ে swap লিখতে পারি
- ✓ C-এর চারটি স্কোপের নাম বলতে পারি
- ✓ স্টোরেজ ক্লাস টেবিল পূরণ করতে পারি
- ✓ রিকার্সিভ আউটপুট ট্রেস করতে পারি
- ✓ fib ট্রিতে কল গুনতে পারি